Laboratorio de Computación Paralela: Metodología PCAM y OpenMP

Simulación Gravitacional N-Body 2D

Jonathan Zacarias (Base, Corrección y Ensamblado)
Hugo Barillas (Paralelo y Experimentos)
June Herrera (Optimización y Concurrencia)

Octubre 2026

Resumen

En este laboratorio se presenta el diseño, desarrollo, paralelización y optimización de una simulación gravitatoria bidimensional de N-cuerpos (N-Body 2D) bajo la metodología sistemática **PCAM** (*Partitioning, Communication, Agglomeration, Mapping*) y el estándar de memoria compartida **OpenMP** en lenguaje C. Se formula la línea base secuencial con complejidad algorítmica $\mathcal{O}(N^2)$ y se establecen mecanismos rigurosos de validación numérica mediante la definición de un *checksum* acumulado determinista con semilla fija (42). El presente documento detalla la fase de particionamiento, la estrategia de desacoplamiento temporal, la corrección numérica de la simulación y las respuestas a las interrogantes de escalabilidad planteadas.

1. Introducción y Línea Base Secuencial (Baseline)

El problema de simulación de N-cuerpos (*N-Body Problem*) modela la evolución dinámica en el tiempo de un conjunto de N partículas que interactúan mutuamente según la Ley de Gravitación Universal. En un espacio euclidiano bidimensional, cada cuerpo $i \in \{0, \ldots, N-1\}$ está caracterizado por su posición $\mathbf{r}_i = (x_i, y_i)$, velocidad $\mathbf{v}_i = (v_{x,i}, v_{y,i})$ y masa $m_i > 0$.

La fuerza gravitatoria neta $\mathbf{F}_i$ que experimenta la partícula i debido a la atracción de las restantes $N-1$ partículas viene dada por:

$$\mathbf{F}_i = \sum_{\substack{j=0 \\ j \neq i}}^{N-1} G \frac{m_i m_j}{\left(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2\right)^{3/2}} (\mathbf{r}_j - \mathbf{r}_i) \tag{1}$$

donde $G = 1{,}0$ representa la constante gravitacional normalizada y $\epsilon^2 = 10^{-9}$ es un factor de suavizado (*softening parameter*) indispensable para evitar singularidades y divisiones por cero cuando la distancia euclidiana entre dos partículas tiende a cero.

A partir de la aceleración resultante $\mathbf{a}_i = \mathbf{F}_i / m_i$, la evolución cinemática se resuelve discretizando el tiempo mediante un integrador de Euler con paso temporal $\Delta t = 0{,}01$ s:

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \mathbf{a}_i(t) \cdot \Delta t \tag{2}$$

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \mathbf{v}_i(t + \Delta t) \cdot \Delta t \tag{3}$$

1.1. Complejidad Algorítmica y Desacoplamiento en Dos Fases

El cálculo directo requiere evaluar $N(N-1) = \mathcal{O}(N^2)$ interacciones por cada paso de tiempo, mientras que la integración física demanda únicamente $\mathcal{O}(N)$ operaciones. Para la configuración de referencia ($N = 5000$ cuerpos, 10 pasos), se computan $2{,}5 \times 10^7$ interacciones por paso, consumiendo más del 99.5 % del tiempo en el cálculo de fuerzas.

La implementación secuencial de referencia (`nbody_secuencial.c` / `src/nbody_seq.c`) estructura cada paso en **dos fases desacopladas**:

1. **Fase 1 (Cómputo de Fuerzas):** Se calculan y acumulan las componentes de fuerza en vectores temporales `fx[i]` y `fy[i]` leyendo exclusivamente las posiciones de los cuerpos.
2. **Fase 2 (Actualización Cinemática):** Se actualizan velocidades y posiciones en un bucle separado de orden $\mathcal{O}(N)$.

Este desacoplamiento garantiza que ninguna partícula altere su estado durante el cómputo de fuerzas de un mismo paso, eliminando dependencias cruzadas de lectura y escritura para la posterior paralelización.

2. Aplicación de la Metodología PCAM

2.1. Partition (Particionamiento) - [Responsable: Jonathan Zacarias]

La fase de partición descompone el cómputo en tareas concurrentes elementales de grano fino. En la simulación N-Body se identificaron dos estrategias principales:

1. **Partición por Partícula / Grano Medio ($\mathcal{O}(N)$ tareas):** Se define la tarea T_i como el cálculo integral de la fuerza neta ejercida sobre el cuerpo i. Cada tarea T_i requiere acceso de lectura sobre el arreglo global de posiciones de todas las partículas, pero escribe *únicamente* en sus acumuladores privados `fx[i]` y `fy[i]`. Esta partición expone N tareas estrictamente independientes y libres de condiciones de carrera por diseño (*race-free*), constituyendo la base de `nbody_paralelo.c`.
2. **Partición por Pares de Interacción / Grano Fino ($\mathcal{O}(N^2/2)$ tareas):** Aprovecha la Tercera Ley de Newton ($\mathbf{F}_{ji} = -\mathbf{F}_{ij}$). Cada tarea $T_{i,j}$ (con $j > i$) calcula una interacción y actualiza simultáneamente los cuerpos i y j. Si bien reduce el número de operaciones a la mitad, introduce escrituras concurrentes sobre acumuladores compartidos, requiriendo sincronización o reducción en variables locales por hilo (`nbody_optimized.c`).

2.2. Communication y Race Conditions - [Responsable: June Herrera]

[Aquí June Herrera describe el análisis de comunicación en memoria compartida, el acceso al bus de memoria/caché y la resolución de condiciones de carrera en nbody_optimized.c mediante acumuladores privados o directivas OpenMP atomic/critical].

2.3. Agglomeration y Mapping - [Responsable: Hugo Barillas]

[Aquí Hugo Barillas detalla el agrupamiento de iteraciones (chunks) y la asignación a hilos mediante las políticas schedule(static), schedule(dynamic) y schedule(guided) con 1, 2, 4 y 8 hilos].

3. Decisiones de Diseño con OpenMP

[Justificación técnica de directivas pragma omp parallel for, cláusulas default(none), shared(), private(), firstprivate() y control del overhead de sincronización].

4. Corrección Numérica y Validación de Resultados

Para verificar la corrección numérica de las implementaciones paralelas frente al baseline secuencial, se formuló el **Checksum acumulado** $C(t)$ al finalizar los 10 pasos temporales:

$$C = \sum_{i=0}^{N-1} (x_i + y_i + v_{x,i} + v_{y,i}) \tag{4}$$

4.1. Análisis de Asociatividad en Aritmética Flotante (IEEE-754)

En la aritmética de coma flotante de doble precisión (`double`), la adición no es asociativa: $(a+b)+c \neq a+(b+c)$.

- En la versión paralela (`nbody_paralelo.c`), como cada hilo ejecuta el bucle interno j en el orden exacto $0 \ldots N-1$, el Checksum resultante debe ser **estrictamente idéntico** al secuencial ($\Delta C = 0$).
- En la versión optimizada (`nbody_optimized.c`), las reducciones parciales alteran el orden cronológico de suma, tolerándose un error numérico relativo:

$$\epsilon_{\text{rel}} = \frac{|C_{\text{opt}} - C_{\text{seq}}|}{|C_{\text{seq}}|} < 10^{-11} \tag{5}$$

  Cualquier desviación superior a 10^{-7} evidencia una condición de carrera activa.

Tabla 1: Validación de corrección numérica con semilla 42 (10 pasos temporales)

Cuerpos (N)	**Checksum Secuencial**	**Checksum Paralelo**	**Error Relativo**	**Estado**
100	Baseline Ref.	Idéntico	$\Delta = 0$	Validado
1000	Baseline Ref.	Idéntico	$\Delta = 0$	Validado
3000	Baseline Ref.	Idéntico	$\Delta = 0$	Validado
5000	Baseline Ref.	Idéntico	$\Delta = 0$	Validado

5. Resultados Experimentales, Speedup y Eficiencia

[Aquí Hugo Barillas inserta la tabla con tiempos promedio (3 repeticiones por configuración), Speedup $S(p) = T_1/T_p$, Eficiencia $E(p) = S(p)/p$ y gráficos comparativos de schedules].

6. Comparación: Versión Paralela vs. Optimizada

[Aquí June Herrera compara el rendimiento de la paralelización $O(N^2)$ frente a la reducción simétrica $O(N^2/2)$ co

7. Conclusiones

[Conclusiones generales sobre la escalabilidad, el impacto de la sincronización en OpenMP y la metodología PCAM].

A. Anexo: Respuestas a Preguntas de Análisis

Pregunta 1: Influencia del tamaño del problema (N) en la relación cómputo/comunicación y en el Speedup [Responsable: Jonathan Zacarias]

Respuesta: *En la simulación N-Body directa, la complejidad computacional del cálculo de fuerzas es cuadrática $\mathcal{O}(N^2)$, mientras que la sobrecarga de comunicación (accesos al bus de memoria compartida, barreras de sincronización y creación de hilos en el modelo fork-join) es de orden $\mathcal{O}(N)$ o proporcional al número de hilos $\mathcal{O}(p)$.*

Para valores pequeños de N ($N \leq 500$), el tiempo de cálculo aritmético es mínimo. En este régimen, la sobrecarga impuesta por el runtime de OpenMP para inicializar los hilos y sincronizarlos al término de cada paso domina sobre el cómputo útil, resultando en un Speedup deficiente o incluso inferior a la unidad ($S < 1$, desaceleración neta).

*Por el contrario, para valores grandes de N ($N \geq 3000$ y $N = 5000$), el número de evaluaciones de fuerza crece hasta $2{,}5 \times 10^7$ operaciones por paso. La relación entre cómputo útil y sobrecarga se incrementa cuadráticamente. Conforme a la **Ley de Amdahl** y la **Ley de Gustafson-Barsis**, la fracción paralelizable del código supera el 99.5 %, amortizando completamente el overhead de OpenMP y logrando un escalado casi lineal para 2, 4 y 8 hilos.*

Pregunta 2: Impacto de la granularidad de la partición y el balance de carga [Responsable: Jonathan Zacarias]

Respuesta: *La granularidad de la partición define el tamaño de las tareas independientes y su grado de uniformidad computacional:*

1. ***En la versión paralela (`nbody_paralelo.c`):*** *Al particionar el bucle externo sobre las partículas i, cada tarea elemental ejecuta exactamente el mismo número de iteraciones ($N-1$ evaluaciones). La carga es estrictamente homogénea. Por ende, la política estática (`schedule(static)`) divide equitativamente las partículas en bloques contiguos de tamaño N/p, logrando un balance de carga óptimo sin incurrir en la sobrecarga de gestión dinámica de colas.*

2. ***En la versión optimizada (`nbody_optimized.c`):*** *Al explotar la simetría mediante pares (i, j) con $j > i$, el cuerpo $i = 0$ evalúa $N - 1$ interacciones, mientras que el cuerpo $i = N - 2$ evalúa una sola interacción. Esto genera una matriz triangular superior con una distribución de carga intrínsecamente desbalanceada. Si se utilizara una partición estática ingenua, los primeros hilos soportarían la mayor parte del trabajo mientras los últimos quedarían ociosos esperando en la barrera. En este escenario, una granularidad más fina combinada con planificaciones dinámicas (`schedule(dynamic, chunk)`) o guiadas (`schedule(guided)`) resulta imprescindible para mitigar el desbalance computacional.*

Preguntas 3, 4, 7 y 8 [Responsable: Hugo Barillas]

[Espacio reservado para las respuestas asignadas a Hugo Barillas].

Preguntas 5 y 6 [Responsable: June Herrera]

[Espacio reservado para las respuestas asignadas a June Herrera].